



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingeniería de Sistemas

INFORME MAQUINA EXPENDEDORA - UNIDAD II

Curso: Programación II

Docente: Breno Solim Luque Zúñiga

Ramirez Ajalla, Angie Belen (2024016759)

**Tacna – Perú
2026**

INFORME MÁQUINA EXPENDEDORA

I. INFORMACIÓN GENERAL

Objetivos

- Aplicar el uso de diccionarios (Dictionary) en C# para almacenar códigos y precios de productos
- Utilizar controles de Windows Forms para interactuar con el usuario
- Implementar eventos y métodos para realizar compras dentro de una máquina expendedora
- Mostrar productos mediante un DataGridView

Equipos materiales programas y recursos utilizados

- Visual Studio
- Lenguaje de programación C#
- Windows Forms (.NET)
- Dictionary
- DataGridView

II. MARCO TEÓRICO

Dictionary en C#

Un Dictionary es una colección genérica de C# que almacena datos mediante una clave y un valor. Cada clave es única y permite acceder rápidamente a la información guardada.

Windows Forms

Windows Forms es una biblioteca de .NET utilizada para desarrollar aplicaciones de escritorio con interfaces gráficas. Permite trabajar con controles como botones cuadros de texto etiquetas y tablas.

DataGridView

El DataGridView es un control de Windows Forms que muestra información en forma de tabla mediante filas y columnas. Facilita la visualización y organización de datos.

III. PROCEDIMIENTO

Código:

Primero se importaron las librerías necesarias para trabajar con formularios diccionarios y controles gráficos en C#. También se declararon variables para almacenar el crédito el precio del producto y el total de ventas realizadas. Después se crearon diccionarios para guardar los nombres y precios de los productos mediante códigos.

```
1  using System;
2  using System.Collections.Generic;
3  using System.Windows.Forms;
4
5  namespace Maquina_Expendedora
6  {
7      3 referencias
8      public partial class Form1 : Form
9      {
10         // VARIABLES GENERALES
11         double credito = 0;
12         double precio = 0;
13         double ventas = 0;
14
15         // DICCIONARIO DE NOMBRES
16         Dictionary<string, string> nombres =
17             new Dictionary<string, string>()
18             {
19                 {"015", "Papas"},
20                 {"020", "Gaseosa"},
21                 {"030", "Chocolate"},
22                 {"040", "Agua"},
23                 {"050", "Galleta"},
24                 {"060", "Energizante"}
25             };
26
27         // LISTA DE PRODUCTOS
28         Dictionary<string, double> productos =
29             new Dictionary<string, double>()
30             {
31                 {"015", 5.00},
32                 {"020", 2.50},
33                 {"030", 3.00},
34                 {"040", 1.50},
35                 {"050", 4.00},
36                 {"060", 6.00}
37             };
38     }
```

Luego se inicializó el formulario y se configuraron los controles principales del sistema. También se cargaron automáticamente los productos dentro del DataGridView para mostrar la lista disponible.

```
1 referencia
38  public Form1()
39  {
40      InitializeComponent();
41
42      // LOS TEXTBOX SOLO MOSTRARÁN DATOS
43      textBox1.ReadOnly = true;
44      textBox2.ReadOnly = true;
45
46      // SOLO 3 DÍGITOS PARA EL CÓDIGO
47      textBox1.MaxLength = 3;
48
49      // CRÉDITO INICIAL
50      label6.Text = "Crédito: S/0.00";
51
52      // MOSTRAR PRODUCTOS EN EL DATAGRIDVIEW
53
54      dataGridView1.Rows.Add("015", "Papas");
55      dataGridView1.Rows.Add("020", "Gaseosa");
56      dataGridView1.Rows.Add("030", "Chocolate");
57      dataGridView1.Rows.Add("040", "Agua");
58      dataGridView1.Rows.Add("050", "Galleta");
59      dataGridView1.Rows.Add("060", "Energizante");
60
61      // BLOQUEAR EDICIÓN
62
63      dataGridView1.ReadOnly = true;
64
65      dataGridView1.AllowUserToAddRows = false;
66  }
```

Después se implementó un método para buscar automáticamente el precio del producto según el código ingresado. El sistema verifica si el código existe dentro del diccionario y luego muestra el precio correspondiente en el segundo TextBox. También se programaron los botones numéricos para permitir el ingreso de códigos dentro de la máquina expendedora. Cada botón agrega un número al TextBox y ejecuta nuevamente la búsqueda del producto.

```
68
69 // MÉTODO PARA BUSCAR PRODUCTO
70
71
72 11 referencias
private void BuscarProducto()
73 {
74     // EL TEXTBOX1 MOSTRARÁ EL CÓDIGO
75     // EL TEXTBOX2 MOSTRARÁ EL PRECIO
76
77     if (productos.ContainsKey(textBox1.Text))
78     {
79         precio = productos[textBox1.Text];
80
81         textBox2.Text =
82             "S/" + precio.ToString("0.00");
83     }
84     else
85     {
86         textBox2.Text = "";
87     }
88 }
89
90
91 // BOTONES NUMÉRICOS
92
93
94 1 referencia
private void btn1_Click(object sender, EventArgs e)
95 {
96     textBox1.Text += "1";
97     BuscarProducto();
98 }
99
100 1 referencia
private void btn2_Click(object sender, EventArgs e)
101 {
102     textBox1.Text += "2";
103     BuscarProducto();
104 }
105
106 1 referencia
private void btn3_Click(object sender, EventArgs e)
107 {
108     textBox1.Text += "3";
109     BuscarProducto();
110 }
111
```

```

111
112     1 referencia
113     private void btn4_Click(object sender, EventArgs e)
114     {
115         textBox1.Text += "4";
116         BuscarProducto();
117     }
118
119     1 referencia
120     private void btn5_Click(object sender, EventArgs e)
121     {
122         textBox1.Text += "5";
123         BuscarProducto();
124     }
125
126     1 referencia
127     private void btn6_Click(object sender, EventArgs e)
128     {
129         textBox1.Text += "6";
130         BuscarProducto();
131     }
132
133     1 referencia
134     private void btn7_Click(object sender, EventArgs e)
135     {
136         textBox1.Text += "7";
137         BuscarProducto();
138     }
139
140     1 referencia
141     private void btn8_Click(object sender, EventArgs e)
142     {
143         textBox1.Text += "8";
144         BuscarProducto();
145     }
146
147     1 referencia
148     private void btn9_Click(object sender, EventArgs e)
149     {
150         textBox1.Text += "9";
151         BuscarProducto();
152     }

```

Después se implementó el botón DEL para borrar números del código ingresado y actualizar nuevamente la búsqueda del producto. También se programó el botón FN para reiniciar completamente el sistema limpiando los TextBox y reiniciando el crédito mostrado en pantalla.

```
155 // BOTÓN DEL
156 1 referencia
157 private void button12_Click(object sender, EventArgs e)
158 {
159     if (textBox1.Text.Length > 0)
160     {
161         textBox1.Text =
162             textBox1.Text.Substring(
163                 0,
164                 textBox1.Text.Length - 1);
165
166         BuscarProducto();
167     }
168 // BOTÓN FN
169
170 1 referencia
171 private void btnFN_Click(object sender, EventArgs e)
172 {
173     // LIMPIAR TODO EL SISTEMA
174
175     textBox1.Clear();
176     textBox2.Clear();
177
178     credito = 0;
179     precio = 0;
180
181     label6.Text = "Crédito: S/0.00";
182
183     MessageBox.Show(
184         "Sistema reiniciado");
185 }
```

Luego se implementaron los botones monetarios para aumentar el crédito disponible del usuario. Cada vez que se presiona una cantidad el sistema actualiza automáticamente el Label del crédito mostrando el saldo acumulado.

```
186 // BOTONES DE MONEDAS
187
188 // S/5.00
189 1 referencia
190 private void button1_Click(object sender, EventArgs e)
191 {
192     credito += 5.00;
193
194     label6.Text =
195         "Crédito: S/" +
196         credito.ToString("0.00");
197 }
198
199 // S/2.00
200 1 referencia
201 private void button2_Click(object sender, EventArgs e)
202 {
203     credito += 2.00;
204
205     label6.Text =
206         "Crédito: S/" +
207         credito.ToString("0.00");
208 }
209
210 // S/1.00
211 1 referencia
212 private void button3_Click(object sender, EventArgs e)
213 {
214     credito += 1.00;
215
216     label6.Text =
217         "Crédito: S/" +
218         credito.ToString("0.00");
219 }
220
221 // S/0.50
222 1 referencia
223 private void button6_Click(object sender, EventArgs e)
224 {
225     credito += 0.50;
226
227     label6.Text =
228         "Crédito: S/" +
229         credito.ToString("0.00");
230 }
```

También se programaron los botones restantes de monedas para seguir aumentando el crédito disponible del usuario. Cada vez que se presiona una cantidad el sistema actualiza automáticamente el saldo mostrado en pantalla.

```
229 // S/0.20
230 1 referencia
231 private void button5_Click(object sender, EventArgs e)
232 {
233     credito += 0.20;
234
235     label6.Text =
236         "Crédito: S/" +
237         credito.ToString("0.00");
238 }
239 // S/0.10
240 1 referencia
241 private void button4_Click(object sender, EventArgs e)
242 {
243     credito += 0.10;
244
245     label6.Text =
246         "Crédito: S/" +
247         credito.ToString("0.00");
248 }
```

Después se programó el botón ENTER para validar si el producto existe y si el usuario posee suficiente crédito para realizar la compra. El sistema también calcula automáticamente el vuelto correspondiente y muestra la información del producto seleccionado.

```
248 // BOTÓN ENTER
249 1 referencia
250 private void btnENTER_Click(object sender, EventArgs e)
251 {
252     // VALIDAR SI EXISTE EL PRODUCTO
253
254     if (productos.ContainsKey(textBox1.Text))
255     {
256         // VALIDAR CRÉDITO
257
258         if (credito >= precio)
259         {
260             double vuelto = credito - precio;
261
262             string producto =
263                 nombres[textBox1.Text];
264
265             MessageBox.Show(
266                 "Producto entregado\n\n" +
267                 "Producto: " + producto +
268                 "\nCódigo: " + textBox1.Text +
269                 "\nPrecio: S/" +
270                 precio.ToString("0.00") +
271                 "\nVuelto: S/" +
272                 vuelto.ToString("0.00"));
273
274             // ACUMULAR VENTAS
275             ventas += precio;
276         }
277     }
278 }
```

Dentro del botón ENTER se programó la limpieza automática del sistema después de realizar una compra correcta. El crédito vuelve a cero se limpia el código ingresado y también el precio mostrado en pantalla. Además, se agregaron mensajes para informar cuando el crédito es insuficiente o cuando el código ingresado no existe.

```
275
276 // LIMPIAR TODO
277 credito = 0;
278
279 label6.Text =
280     "Crédito: S/0.00";
281
282 textBox1.Clear();
283 textBox2.Clear();
284 }
285 else
286 {
287     MessageBox.Show(
288         "Crédito insuficiente");
289 }
290
291 else
292 {
293     MessageBox.Show(
294         "Código incorrecto");
295 }
296 }
297
```

Finalmente se implementó el botón CHANGE, devuelve el cambio restante al usuario el botón, también el botón EXIT que cierra completamente la aplicación y el botón REPORTE muestra el total acumulado de ventas realizadas por la máquina expendedora.

```
298 // BOTÓN CAMBIO
299
300
301 1 referencia
302 private void btnCHANGE_Click(object sender, EventArgs e)
303 {
304     MessageBox.Show(
305         "Cambio entregado: S/" +
306         credito.ToString("0.00"));
307
308     credito = 0;
309
310     label6.Text =
311         "Crédito: S/0.00";
312 }
313
314 // BOTÓN EXIT
315
316
317 1 referencia
318 private void btnEXIT_Click(object sender, EventArgs e)
319 {
320     Application.Exit();
321 }
322
323 // BOTÓN REPORTE
324
325
326 1 referencia
327 private void btnREPORTE_Click(object sender, EventArgs e)
328 {
329     MessageBox.Show(
330         "REPORTE DE VENTAS\n\n" +
331         "Ventas totales: S/" +
332         ventas.ToString("0.00"));
333 }
```

```

333 |
334 | 1 referencia
335 | private void dataGridView1_CellContentClick(
336 |     object sender,
337 |     DataGridViewCellEventArgs e)
338 | {
339 | }
340 | }
341 |

```

Ventana:



	CODIGO	PRODUCTO
*		

SELECCIÓN DE COMPRA

Código:

Precio:

HACER SELECCIÓN

1	2	3	FN
4	5	6	CHANGE
7	8	9	EXIT
.	0	DEL	ENTER

CANTIDAD MONETARIA

S/5.00	S/2.00	S/1.00
S/0.50	S/0.20	S/0.10

Crédito: S/0.00

Ejecución:



SELECCIÓN DE COMPRA

Código: 015

Precio: S/5.00

HACER SELECCIÓN

1

2

3

FN

4

5

6

CHANGE

7

8

9

EXIT

.

0

DEL

ENTER

	CODIGO	PRODUCTO
▶	015	Papas
	020	Gaseosa
	030	Chocolate
	040	Agua
	050	Galleta
	060	Energizante

CANTIDAD MONETARIA

S/5.00

S/2.00

S/1.00

S/0.50

S/0.20

S/0.10

Crédito: S/5.00

REPORTE

✕

Producto entregado

Producto: Papas

Código: 015

Precio: S/5,00

Vuelto: S/0,00

Aceptar



SELECCIÓN DE COMPRA

Código:

Precio:

HACER SELECCIÓN

1

2

3

FN

4

5

6

7

8

9

.

0

DEL

	CODIGO	PRODUCTO
▶	015	Papas
	020	Gaseosa
	030	Chocolate
	040	Agua
	050	Galleta
	060	Energizante

CANTIDAD MONETARIA

S/5.00

S/2.00

S/1.00

S/0.50

S/0.20

S/0.10

Crédito: S/0.00

REPORTE

✕

Sistema reiniciado

Aceptar

IV. ANÁLISIS E INTERPRETACIÓN DE RESULTADOS

¿Qué indican los resultados?

- Los resultados muestran que el uso de diccionarios y controles de Windows Forms facilita el desarrollo de sistemas interactivos para compras automáticas.
- La aplicación permite controlar productos crédito y ventas de manera ordenada y sencilla.

¿Qué se ha encontrado?

- Se comprobó que los diccionarios permiten almacenar información de productos eficientemente
- Se observó que el DataGridView mejora la visualización de los productos disponibles
- Se evidenció que los eventos de botones permiten una interacción dinámica con el usuario

CONCLUSIONES

- El trabajo permitió aprender el uso de diccionarios en C# para relacionar códigos productos y precios
- Se aprendió a utilizar controles de Windows Forms para desarrollar interfaces gráficas interactivas
- El programa funcionó correctamente permitiendo realizar compras controlar crédito y mostrar cambio automáticamente
- El uso de DataGridView facilitó la visualización de productos disponibles dentro de la máquina expendedora

RECOMENDACIONES

- Verificar que los nombres de los controles coincidan con los utilizados en el código
- Configurar correctamente los eventos de botones para evitar errores de ejecución
- Validar los códigos ingresados antes de procesar una compra
- Probar todas las funciones del sistema antes de finalizar el proyecto

WEBGRAFIA

Microsoft. (s.f.). Dictionary<TKey,TValue> Class. Microsoft Learn. <https://learn.microsoft.com/es-es/dotnet/api/system.collections.generic.dictionary-2>

Microsoft. (s.f.). Windows Forms overview. Microsoft Learn. <https://learn.microsoft.com/es-es/dotnet/desktop/winforms/overview/>

Microsoft. (s.f.). DataGridView Control. Microsoft Learn. <https://learn.microsoft.com/es-es/dotnet/desktop/winforms/controls/datagridview-control-windows-forms>